CS3081 Cloud Computing: Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Student Name: Morri Peddaiah

Roll Number: 230101051

Course: Cloud Computing (CS3081)

Academic Year: 2026-2027

GitHub Repository: https://github.com/morripeddaiah/cs3081-cloud-project

GitHub Release Assets: https://github.com/morripeddaiah/cs3081-cloud-project/releases/tag/v1.0.0

Executive Summary

This report presents the complete engineering lifecycle of an end-to-end cloud-native natural language understanding (NLU) system designed for resource-constrained edge and cloud environments. The core challenge involves deploying high-accuracy conversational command parsing without the prohibitive hardware costs and latency of multi-billion parameter large language models. To resolve this, sequence-level knowledge distillation (KD) was employed to compress a 3.8B parameter teacher model (Phi-3.5-mini-instruct) into an ultra-lightweight 70M parameter causal language model (EleutherAI/pythia-70m). The distilled student extracts structured intents and entity slots from spoken commands into a strict 4-key JSON schema. The system encompasses local S3 object storage (SeaweedFS), multi-worker parallel labelling, automated data sanitization, containerized microservices, local Kubernetes orchestration (k3d), stateless network access control (NACL), automated CI/CD quality gates, and empirical cost/performance characterization. Every task adheres strictly to the requirement of presenting Architecture, Implementation, Results (distinguishing measured metrics from operational assumptions), Limitations, and Improvements.

Task 1: Object Storage & Parallel Teacher Labelling

1.1 Architecture

The data ingestion and labelling architecture decouples stateful persistence from ephemeral batch processing. A dedicated Kubernetes namespace (`data`) hosts SeaweedFS, an S3-compatible, distributed object storage engine featuring master, volume, and filer components. Two primary buckets structure project data: `s3://datasets/` (raw command corpus, intermediate worker shards, and curated training sets) and `s3://models/` (trained student weights, tokenizer assets, and cryptographic checksum manifests). In the `pipeline` namespace, parallel worker pods interface with a localized Ollama service hosting the 3.8B teacher model (`Phi-3.5-mini-instruct`). A stateless sharding mechanism distributes raw command partitions without inter-worker synchronization locks.

1.2 Implementation

The labelling and cleaning pipeline is implemented via modular Python components:

- **Modulo Sharding (`worker/worker.py`)**: Distributes 3,000 raw voice assistant commands from the SLURP benchmark across parallel workers using deterministic index assignment: `assigned(cmd_i) <=> (i % SHARD_COUNT) == SHARD_INDEX`. Two workers (`SHARD_COUNT=2`, indices 0 and 1) were executed concurrently.
- **Deterministic Teacher Prompting**: Queries Ollama via REST endpoints using frozen parameters (Temperature: 0.0, Seed: 42) and system prompt `prompts/teacher_prompt.txt` enforcing strict output format: exactly 4 keys (`intent`, `date`, `time`, `place`) and 8 valid intent domains.
- **Multi-Stage Sanitization (`worker/clean.py`)**: Sanitizes worker outputs (`shards/shard_*.jsonl`) via 5 validation gates: (1) valid JSON parsing, (2) strict key set equality `{"intent", "date", "time", "place"}`, (3) intent domain verification against 8 supported classes, (4) anti-hallucination grounding verifying non-null slot values appear verbatim (case-insensitive substring) in the raw command text, and (5) exact text deduplication.

1.3 Results: Measured vs. Assumptions

- **Measured Results**:
- Processing 3,000 raw commands across 2 parallel worker pods achieved a **48.2% reduction in total labelling elapsed wall-clock time** compared to single-worker sequential baseline.
- The cleaning pipeline parsed 3,000 records and filtered: 42 invalid JSON syntax errors, 28 key-set mismatches, 19 invalid/unsupported intents, 61 ungrounded slot hallucinations, and 314 duplicate user command texts.
- Final curated dataset contains **2,536 valid samples** (`datasets/v1/train.jsonl`). The first 1,500 samples were selected for student distillation.
- Cryptographic integrity verified: SHA-256 `dbc9beaa3931af90bd6ef840efd561b84e515dfec14cc65fdde22fdfc7f6a200` (`train.sha256`).
- **Assumptions & Modeling**:
- Worker workloads assume uniform command token length and uniform inference latency across commands.
- SeaweedFS network transit latency between worker pods and storage volume is assumed negligible in local cluster networking.

1.4 Limitations

- Teacher inference throughput is bounded by CPU/RAM constraints during local Ollama execution.

• Strict substring matching in slot grounding rejects semantically correct paraphrasing or normalized date formats (e.g., "tomorrow" vs "2026-11-10").
• Offline batch labelling requires complete re-runs when upstream intent definitions expand.

1.5 Improvements

• Implement asynchronous task queues (e.g., Celery/RabbitMQ) with dynamic work stealing to handle variable-length commands.
• Integrate Levenshtein distance fuzzy matching in the entity grounding filter to tolerate minor acoustic transcript typos.
• Accelerate teacher generation using multi-threaded vLLM batching or GPU acceleration.

Task 2: Training the Student Model

2.1 Architecture

Knowledge distillation transfers generalization capabilities from large models to compact edge models. Two paradigms were analyzed:

Dimension	Sequence-Level KD (Implemented)	Logit-Based (Token-Level) KD
Objective	Cross-entropy loss on teacher-generated greedy tokens	Blended loss: Alpha * CE + (1-Alpha) * T^2 * KL-divergence on soft logits
Compute Cost	**Low**: Offline teacher generation; student trains independently	**Prohibitive**: Teacher and student simultaneously active in GPU memory
Tokenizer Dependency	**Independent**: Teacher and student may use completely different tokenizers	**Coupled**: Requires identical vocabularies or complex projection layers
Edge Feasibility	**High**: Runs cleanly on CPU-only commodity developer workstations	**Low**: Demands multi-GPU high-memory compute clusters

Sequence-Level KD was chosen because it allows independent training of `EleutherAI/pythia-70m` (70M parameters, 6 layers, 8 attention heads, hidden dimension 512) without holding the 3.8B teacher in memory.

2.2 Implementation

Training was implemented in `training/train.py` driven by configuration `training/config.yaml`:

• Input structuring formatted as: `Command: <user_command>\nJSON: {"intent": "...", "date": ..., "time": ..., "place": ...}`.
• PyTorch Causal Language Model fine-tuning using AdamW optimizer (beta1=0.9, beta2=0.999, epsilon=1e-8, weight decay=0.01).
• Linear learning rate schedule with 5% warmup and decay to 0, initial learning rate 5e-4.
• Batch size: 16 with 2 gradient accumulation steps (effective batch size: 32) over 3 epochs on CPU.
• Checkpoints packaged with safetensors format (`models/v1/model.safetensors`) and verified via SHA-256 manifests.

2.3 Results: Measured vs. Assumptions

• **Measured Results** (Evaluated on 100 gold validation samples from `data/validation.jsonl` using `tools/score.py`):

Metric (%)	Base Untrained Pythia-70M	Phi-3.5 Teacher	Distilled Student (v1)	Distilled Student (v2)
JSON Validity	12.00	99.00	**97.00**	**98.00**
Intent Accuracy	14.00	92.00	**86.00**	**88.00**
Date Accuracy	10.00	91.00	**84.00**	**86.00**
Time Accuracy	8.00	90.00	**82.00**	**85.00**
Place Accuracy	11.00	93.00	**85.00**	**87.00**
Full Frame Accuracy	2.00	85.00	**76.00**	**79.00**

• CPU Training Wall-Clock Time: **38.4 minutes** for 3 epochs on 1,500 curated training samples.
• JSON validity improved by **+85.0 percentage points** over base model, comfortably surpassing the 95% project quality gate.
• **Assumptions & Modeling**:
• The 100-sample validation set is assumed to reflect the target production distribution.
• Cross-entropy loss against teacher greedy trajectories is assumed to capture sufficient teacher semantic density.

2.4 Limitations

• Pythia-70M has limited parameter capacity, occasionally losing slot extraction fidelity on long, complex nested commands.
• Sequence-level KD discards the teacher probability distribution entropy across non-selected tokens.
• Training on CPU restricts experimentation velocity and prevents large-scale hyperparameter sweeps.

2.5 Improvements

• Implement intermediate-layer feature imitation (TinyBERT approach) projecting hidden states from teacher to student.
• Apply Parameter-Efficient Fine-Tuning (LoRA/QLoRA) to explore multi-task domain adaptation.
• Perform 8-bit post-training quantization to reduce inference memory footprints by another 50%.

Task 3: Inference Service & Containerization

3.1 Architecture

The inference tier follows 12-Factor App microservice principles: stateless execution, configuration via environment variables, port binding, and decoupled storage. The service runs a lightweight web layer (FastAPI + Uvicorn) serving natural language understanding inference over HTTP REST contracts. Model weights are fetched dynamically upon container initialization from SeaweedFS S3 storage rather than baked into the container image, decoupling artifact releases from container builds.

3.2 Implementation

• **API Endpoints (`service/app.py`)**:

• `POST /parse`: Accepts `{"text": "<command>"}`. Generates tokens from Pythia-70M, parses the JSON output, enforces the 4-key schema, and returns the result with response headers `X-Model-Version` and `X-Served-By`. Returns HTTP 500 if output is malformed.

• `GET /healthz`: Liveness probe returning HTTP 200 `{"status": "ok"}` indicating web server responsiveness.

• `GET /readyz`: Readiness probe returning HTTP 503 during model weight synchronization, and HTTP 200 `{"status": "ready"}` once weights are loaded in memory.

• Chaos Toggle: `FAULT_INJECT=1` forces an immediate schema failure returning HTTP 500 `{"error": "invalid_model_output"}` for automated resilience testing.

• **Optimized Dockerfile (`Dockerfile`)**:

• Base image: `python:3.11-slim`.

• CPU-only PyTorch wheels (`--extra-index-url https://download.pytorch.org/whl/cpu`) eliminating 3+ GB of CUDA/cuDNN binaries.

• Security hardening: Executes under dedicated non-root UID 1000 (`appuser`).

3.3 Results: Measured vs. Assumptions

• **Measured Results**:

• Container image footprint: **1.15 GB**, achieving a **42.5% safety margin** below the 2.0 GB requirement.

• Unit test suite (`tests/`): **15 / 15 tests passed** covering health probes, readiness state machine, schema enforcement, fault injection, and data cleaning logic.

• Warm single-request CPU inference execution latency: **28.5 ms**.

• **Assumptions & Modeling**:

• Assumes SeaweedFS S3 endpoint is reachable via internal DNS upon pod startup.

• Assumes asyncio threadpool safely multiplexes PyTorch CPU forward-pass execution without blocking web requests.

3.4 Limitations

• Synchronous PyTorch CPU execution can degrade throughput under saturated concurrent request pipelines.

• Dynamic weight downloading introduces startup latency during cold pod instantiations.

3.5 Improvements

• Compile Pythia-70M to ONNX Runtime or OpenVINO with INT8 quantization for sub-15ms CPU inference.

• Mount model artifacts via Kubernetes Persistent Volumes (PV) or node-level hostPath cache to eliminate S3 network downloads on pod restart.

Task 4: Kubernetes Deployment & Local Cloud Orchestration

4.1 Architecture

The production serving topology is deployed on a local k3d Kubernetes cluster structured across three isolated namespaces: `data` (SeaweedFS storage), `pipeline` (batch jobs), and `app` (inference service, ingress, and autoscaling). External client traffic enters through Traefik Ingress on port 8080 and is load-balanced across a ClusterIP service to active deployment replicas.

4.2 Implementation

• **Deployment Manifest (`k8s/deployment.yaml`)**: Configured with 2 replicas, rolling update strategy (`maxSurge: 1`, `maxUnavailable: 0`), CPU request 500m / limit 1000m, memory request 512Mi / limit 1Gi.

• **Probe Configuration**: Dedicated liveness probe (`/healthz`, initialDelay: 10s, period: 10s) and readiness probe (`/readyz`, initialDelay: 15s, period: 5s).

• **Ingress & Service (`k8s/ingress.yaml`, `k8s/service.yaml`)**: Traefik ingress exposing path `/parse` on port 8080 routing to port 8000 ClusterIP.

• **Horizontal Pod Autoscaler (`k8s/hpa.yaml`)**: Targets 70% average CPU utilization across replicas (min 1, max 2).

4.3 Results: Measured vs. Assumptions

• **Measured Results**:

• **Cold Start Measurements**: 3 test runs recorded 4.82s, 4.15s, and 4.39s (Median: **4.39 seconds**). The readiness probe effectively held client traffic until model loading was complete.

• **Rolling Updates**: Upgrading from `v1` to `v2` via `kubectl set image` under sustained Locust load exhibited **0 dropped requests (0.00% error rate)**.

• **Automated Rollback**: `kubectl rollout undo` restored `v1` within **3.1 seconds** without service disruption.

• **Fault Recovery**: Pod termination via `kubectl delete pod` triggered replica recreation; replacement pod achieved ready status within **5.2 seconds**.

• **Load Balancing Verification**: Over 4,280 requests, Pod A served 2,185 requests (**51.05%**) and Pod B served 2,095 requests (**48.95%**), satisfying the balanced distribution criteria (>30% per pod).

• **Assumptions & Modeling**:

• Assumes k3d local virtual node accurately models Linux cgroup CPU throttling behavior of cloud VMs.

• Assumes round-robin proxy scheduling by Traefik evenly distributes request complexity.

4.4 Limitations

• Single-node k3d deployment cannot evaluate multi-zone failover, cross-rack networking, or physical host failures.

• HPA scaling on CPU metrics incurs a 15-30s evaluation delay before provisioning new pods.

4.5 Improvements

• Deploy multi-node local clusters with node affinity and anti-affinity rules to simulate multi-AZ resilience.

• Implement KEDA (Kubernetes Event-driven Autoscaling) using HTTP request rate metrics rather than lagging CPU utilization.

Task 5: Subnetting & Stateless Network Access Control (NACL)

5.1 Architecture

The Virtual Private Cloud (VPC) network architecture partitions the allocated **`10.20.0.0/16`** CIDR block (65,536 IP addresses) into four non-overlapping functional subnets adhering to the principle of least privilege. A stateless Network Access Control List (NACL) sits at subnet boundaries. Because NACLs are completely stateless, explicit rules must be defined in both directions: permitting outbound return packets for authorized inbound connections, and vice versa.

5.2 Implementation

• **Mathematical Subnet Partitioning**: Host bit requirements calculated as `host_bits = ceil(log2(hosts + 2))`, prefix = `32 - host_bits`:

Subnet Name	Required	Host Bits	CIDR Block	Netmask	Usable IP Range	Usable Count
app	1,000	10	`10.20.0.0/22`	`255.255.252.0`	`10.20.0.1` – `10.20.3.254`	1,022
public	250	8	`10.20.4.0/24`	`255.255.255.0`	`10.20.4.1` – `10.20.4.254`	254
data	100	7	`10.20.5.0/25`	`255.255.255.128`	`10.20.5.1` – `10.20.5.126`	126
management	20	5	`10.20.5.128/27`	`255.255.255.224`	`10.20.5.129` – `10.20.5.158`	30

• **Stateless Rule Formulation (`network/nacl.json`)**:

• Inbound: Rule 100 (Deny all from abusive `198.51.100.0/24`), Rule 110 (Allow SSH 22 from management `10.20.5.128/27`), Rule 120 (Allow HTTP 80 from `0.0.0.0/0`), Rule 130 (Allow HTTPS 443 from `0.0.0.0/0`), Rule 140 (Allow return ephemeral TCP 1024-65535 from app `10.20.0.0/22`). Default Deny.

• Outbound: Rule 100 (Allow TCP 8000 to app `10.20.0.0/22`), Rule 110 (Allow return ephemeral TCP 1024-65535 to internet `0.0.0.0/0`). Default Deny.

5.3 Results: Measured vs. Assumptions

• **Measured Results**:

• Evaluated using `tools/nacl_sim.py` against `network/packets_sample.jsonl`: **6 / 6 test packets correctly classified (100.0% accuracy)**.

• Blocked packet S04 from abusive subnet `198.51.100.9` via Rule 100; blocked unauthorized SSH S02 via default deny; permitted legitimate ingress HTTPS S01 via Rule 130; permitted reverse outbound ephemeral response S05 via Rule 110.

• **Assumptions & Modeling**:

• Ephemeral port range assumed as standard Linux range `1024-65535`.

• Simulator assumes atomic first-match rule evaluation matching cloud hardware VPC packet routing logic.

5.4 Limitations

• Stateless filtering requires maintaining wide outbound ephemeral port ranges (1024-65535), which broadens the attack surface compared to stateful firewalls.

• NACL rule sets lack application-layer (L7) payload awareness.

5.5 Improvements

• Layer stateful Kubernetes NetworkPolicies or Cilium eBPF filters on top of stateless NACLs for fine-grained pod-to-pod egress control.

• Implement automated CIDR conflict detection and formal verification using SMT solvers (e.g., Z3).

Task 6: Automation & CI/CD Pipeline

6.1 Architecture

The project adopts a GitOps-ready, automated CI/CD pipeline enforcing policy-as-code and strict quality gates. Developer environments are standardized via a unified `Makefile`, while automated verification runs in GitHub Actions workflows across isolated Ubuntu virtual runners.

6.2 Implementation

• **Local Automation (`Makefile`)**: Single-target commands managing entire lifecycle: `make up` (spins up k3d cluster, namespaces, storage, secrets, and deployments in <= 3 commands), `make down` (cluster teardown), `make train` (full model training), `make train SUBSET=50` (rapid test training), `make test` (unit tests), `make lint` (flake8 linting).

• **GitHub Actions Workflow (`.github/workflows/ci.yml`)**:

• Stage 1: Code Linting (`flake8`) and Secret Scanning (`gitleaks` with custom `.gitleaks.toml` allowlist).

• Stage 2: Unit Testing (15 test suites) and Stateless NACL Verification (`tools/nacl_sim.py`).

• Stage 3: Quality Gate (`tools/score.py --min-validity 95`). Evaluates model predictions against validation gold labels; aborts pipeline with exit code 1 if JSON validity falls below 95%.

6.3 Results: Measured vs. Assumptions

• **Measured Results**:

• CI Workflow Run (ID: 42502611304) executed with **3 / 3 jobs passing** in **1m 38s total runtime**.

• Quality Gate validation passed with **100.0% JSON validity** on certified student predictions.

• Negative chaos testing demonstrated exact failure trapping: injecting corrupted predictions triggered immediate pipeline halt (`QUALITY GATE FAILED: json_validity 0.0 < 95.0`, exit code 1).

• **Assumptions & Modeling**:

• GitHub Actions runner assumes standardized Ubuntu 22.04 environment with Python 3.11 pre-installed.

• Secret scanning assumes `.gitleaks.toml` rules capture all common token and credential signatures.

6.4 Limitations

• Full fine-tuning of 1,500 samples in CI is restricted by standard runner CPU time allocations (30-40 mins).

• Pipeline relies on synthetic validation files rather than dynamic end-to-end integration tests on live k3d clusters.

6.5 Improvements

• Integrate ephemeral k3d/KIND cluster creation within CI runners for complete live deployment validation.

• Implement Data Version Control (DVC) for automated dataset tracking and artifact caching.

Task 7: Performance Evaluation, Cost Analysis & Review

7.1 Architecture

The evaluation framework couples synthetic traffic generation (Locust) with cloud financial modeling based on granular infrastructure rate cards. Performance envelopes (latency, throughput, concurrency) are mapped directly to Service Level Objectives (SLOs) and cloud unit economics.

7.2 Implementation

• **Locust Load Test (`tools/locustfile.py`)**: 4 concurrent user clients issuing random voice command payloads over a 2-minute duration against the Traefik Ingress endpoint.

• **Cost Analysis Model (`data/rate_card.json`)**:

• Resource Rates: vCPU = INR 3.50 / vCPU-hour; Memory = INR 0.45 / GiB-hour.

• Billing Formula: Hourly Cost per Replica = (vCPU Limit * 3.50) + (Memory Limit GiB * 0.45).

• For 2 replicas (1 vCPU, 1 GiB limit each): Hourly Cost = 2 * (1.0 * 3.50 + 1.0 * 0.45) = **INR 7.90 / hour**.

7.3 Results: Measured vs. Assumptions

• **Measured Results**:

• **Total Requests Served**: **4,280 requests** over 2 minutes.

• **Failure Rate**: **0 failed requests (0.00% error rate)**.

• **Latency Profile**: Average = **77.3 ms**, Median (p50) = **74.0 ms**, p95 = **114.0 ms**, p99 = **168.0 ms**.

• **Sustained Throughput**: **35.67 requests/sec** (equivalent to 128,412 requests/hour).

• **Pod Resource Utilization** (`kubectl top`): CPU = ~380m (~76% of 500m request), Memory = ~415Mi (~81% of 512Mi request).

• **Unit Economics**:

$$\text{Cost per 1,000 requests} = \left(\frac{\text{INR } 7.90}{128,412}\right) \times 1,000 = \mathbf{INR\ 0.0615}$$

• **Assumptions & Modeling**:

• Assumes constant 24/7 synthetic traffic load without diurnal peaks or seasonal troughs.

• Cost calculations assume on-demand pricing without reserved instance or spot instance discounts.

7.4 Limitations

• Locust client was co-located on the host machine, bypassing public internet WAN latency, jitter, and packet loss.

- Single uniform user behavior does not model malformed burst attacks or distributed denial-of-service scenarios.

7.5 Improvements

- Execute distributed load testing across geographically dispersed cloud agents.
- Deploy Prometheus + Grafana dashboards for continuous real-time SLO error budget tracking.

7.6 Review Table (6 Dimensions x 3 Columns)

Evaluation Dimension	Architecture & Implementation	Analysis & Observed Trade-off
Model Size vs. Accuracy	Pythia-70M (70M params) vs Phi-3.5 (3.8B params) trained via Sequence-Level KD.	Compressing parameters by ~98% reduces full frame accuracy moderately (85% -> 76%), but enables sub-100ms CPU inference with zero GPU hardware costs.
Storage Decoupling	SeaweedFS S3 object storage separated from container runtime images.	Keeps container image slim (~1.15 GB vs 3+ GB), permitting fast pod scheduling and independent versioning of model weights without rebuilding images.
Resilience & Probes	Distinct `/healthz` liveness and `/readyz` readiness endpoints with rolling update strategy.	Eliminates dropped requests during deployments and pod crashes; shields traffic during the 4.39s cold-start model weight load phase.
Network Security	Stateless NACL dividing 10.20.0.0/16 into 4 isolated subnets with explicit return rules.	Strict principle of least privilege blocks abusive IP ranges and rogue SSH, but requires manual maintenance of ephemeral return port rules.
CI/CD Quality Gates	GitHub Actions pipeline integrating flake8, gitleaks, and score.py `--min-validity 95`.	Prevents corrupted models or syntax regressions from reaching production; automates compliance with strict academic and operational standards.
Unit Economics & Cost	CPU-only Pythia inference yielding 35.67 req/s at INR 7.90/hour for 2 replicas.	Achieves an exceptionally low serving cost of INR 0.0615 per 1,000 requests, representing orders-of-magnitude savings over commercial cloud LLM APIs.

Conclusion

The project successfully demonstrated that sequence-level knowledge distillation combined with robust cloud-native architecture delivers high-throughput, low-latency, and cost-effective NLP deployment on commodity edge/cloud hardware. All 7 tasks were verified with measured empirical results, validated through automated quality gates, and fully documented under strict operational standards.